ICR Power Analysis

Adam Kiehl

2025-10-15

Contents

1 Overview **1**

2 Shared settings and helper functions **2**

3 Jaccard agreement simulation **4**
3.1 Model and assumptions 4
3.2 One Jaccard data-generating function 5

4 Question 1: AI effect on Jaccard agreement **7**

5 Question 2: Between-group Jaccard agreement **9**

6 Coding-time simulation **11**
6.1 Model and assumptions 11
6.2 One coding-time data-generating function 12

7 Question 3: AI effect on coding time **15**

8 Question 4: Between-group coding time **17**

1 Overview

This document estimates power across a range of record counts for four prespecified questions:

1. Does marginal Jaccard agreement differ between AI-assisted and manual coding?
2. Do marginal Jaccard agreement values differ among coder groups?
3. Does marginal coding time differ between AI-assisted and manual coding?
4. Does coding time differ among coder groups?

The simulation assumptions remain pre-hoc. The fitted models and inferential targets match the study analysis:

- Jaccard agreement is simulated directly at the record-by-group level and analyzed with a beta mixed-effects model.
- Coding time is analyzed on the log scale with coder and record random intercepts.
- Both fitted models include the AI-by-group interaction and adjust for text length.
- The time model also adjusts for the uncentered global coding order within coder.
- Marginal contrasts are obtained with `emmeans`; Tukey-adjusted p-values are used for the three Jaccard group comparisons.
- The plots retain the record-count sensitivity-curve approach and mark 500 records as a reference only.

2 Shared settings and helper functions

```r
# Monte Carlo settings
reps <- 1000
alpha <- 0.05
N_min <- 10
N_max <- 600
steps <- 10
N_grid <- seq(N_min, N_max, by = steps)
reference_N <- 500

# Reproducibility
set.seed(42)

# Actual coder roster: 3 clinicians, 2 coders, and 3 students.
coder_roster <- tibble::tribble(
  ~coder_id, ~group,
  "Clinician_1", "Clinician",
  "Clinician_2", "Clinician",
  "Clinician_3", "Clinician",
  "Coder_1", "Coder",
  "Coder_2", "Coder",
  "Student_1", "Student",
  "Student_2", "Student",
  "Student_3", "Student"
) |>
  mutate(
    group = factor(group, levels = c("Clinician", "Coder", "Student"))
  )

# Pre-hoc record-level text-length distribution. Values are not centered.
text_length_mean <- 1500
text_length_sd <- 1000
text_length_min <- 25

simulate_text_length <- function(N) {
  pmax(
    round(rnorm(N, mean = text_length_mean, sd = text_length_sd)),
    text_length_min
  )
}

# Assign AI/manual condition as evenly as possible, including for odd N.
assign_condition <- function(N) {
  n_ai <- floor(N / 2)
  n_manual <- N - n_ai

  sample(c(rep("AI", n_ai), rep("Manual", n_manual))) |>
    factor(levels = c("Manual", "AI"))
}

clip01 <- function(x, eps = 1e-6) {
  pmin(pmax(x, eps), 1 - eps)
}
```

```r
# Common plot annotations.
add_power_references <- function(plot_object) {
  plot_object +
    geom_hline(
      yintercept = 0.80,
      linetype = "dashed",
      color = "red"
    ) +
    geom_vline(
      xintercept = reference_N,
      linetype = "dotted",
      color = "grey35"
    ) +
    annotate(
      "text",
      x = reference_N,
      y = 0.03,
      label = "500 records",
      angle = 90,
      hjust = 0,
      vjust = -0.4,
      color = "grey35",
      size = 3.5
    ) +
    scale_y_continuous(
      limits = c(0, 1),
      labels = scales::percent_format(accuracy = 1)
    ) +
    theme_minimal(base_size = 12)
}

# Report the first grid value at which a power curve reaches 80%.
report_minimum_N <- function(results, question_label) {
  results |>
    group_by(across(any_of("contrast"))) |>
    summarise(
      minimum_N = if (any(power >= 0.80, na.rm = TRUE)) {
        min(N[power >= 0.80], na.rm = TRUE)
      } else {
        NA_integer_
      },
      .groups = "drop"
    ) |>
    mutate(question = question_label, .before = 1)
}
```

```r
# Leave at least one logical core available for the operating system.
available_cores <- parallelly::availableCores()
n_workers <- min(4L, parallelly::availableCores() - 1L)

future::plan(
  future::multisession,
  workers = n_workers
```

```
)

# Prevent nested numerical-library threading inside each R worker.
# Otherwise, each worker may attempt to use multiple CPU threads.
Sys.setenv(
  OMP_NUM_THREADS = "1",
  OPENBLAS_NUM_THREADS = "1",
  MKL_NUM_THREADS = "1",
  VECLIB_MAXIMUM_THREADS = "1"
)

# Use the faster Satterthwaite method for lmer-based emmeans calls.
emmeans::emm_options(
  lmer.df = "satterthwaite",
  disable.pbkrtest = TRUE
)

message(
  "Using ", n_workers,
  " parallel workers out of ",
  available_cores,
  " available cores."
)

parallel_options <- furrr::furrr_options(
  seed = TRUE,
  scheduling = Inf,
  packages = c(
    "tidyverse",
    "lme4",
    "lmerTest",
    "glmmTMB",
    "emmeans"
  )
)

# Configure progress bars for parallel furrr jobs.
# Use progressr's built-in text handler for compatibility across versions.
options(progressr.enable = TRUE)
progressr::handlers("txtprogressbar")
```

3 Jaccard agreement simulation

3.1 Model and assumptions

For record r and coder group g, the simulation generates one mean pairwise Jaccard outcome directly:

$$Y_{rg} \sim \text{Beta}(\mu_{rg}\phi, (1-\mu_{rg})\phi),$$

with

$$\text{logit}(\mu_{rg}) = \text{logit}(p_{rg}) + \beta_L L_r + u_r, \qquad u_r \sim N(0, \sigma^2_{record}).$$

Here, p_{rg} is the prespecified group- and condition-specific mean before the record random effect, L_r is the uncentered text length, and ϕ is the beta precision parameter. The pre-hoc script did not specify a text-length effect, so `beta_text_jaccard` is set to zero during data generation while text length remains in the fitted model.

The same boundary adjustment used in the study analysis is applied after simulation:

$$Y^* = \frac{Y(n-1) + 0.5}{n},$$

where n is the number of record-by-group observations in that simulated dataset.

Each simulated dataset is fitted with:

```
jaccard_beta ~ condition * group + text_length + (1 | record)
```

using `glmmTMB(..., family = beta_family(link = "logit"))`.

```
# Pre-hoc manual-condition Jaccard means.
mu_manual <- c(
  Student = 0.58,
  Clinician = 0.60,
  Coder = 0.62
)

# Pre-hoc beta-model assumptions.
sd_record_jaccard <- 0.50
phi <- 1.5
beta_text_jaccard <- 0
```

3.2 One Jaccard data-generating function

```
simulate_jaccard_data <- function(N, delta_AI) {
  record_data <- tibble(
    record = factor(seq_len(N)),
    condition = assign_condition(N),
    text_length = simulate_text_length(N),
    u_record = rnorm(N, mean = 0, sd = sd_record_jaccard)
  )

  out <- tidyr::crossing(
    record = factor(seq_len(N), levels = levels(record_data$record)),
    group = factor(
      c("Clinician", "Coder", "Student"),
      levels = c("Clinician", "Coder", "Student")
    )
  ) |>
    left_join(record_data, by = "record") |>
    mutate(
      p_manual = unname(mu_manual[as.character(group)]),
      p_mean = if_else(
        condition == "AI",
        p_manual + delta_AI,
        p_manual
      ),
      p_mean = clip01(p_mean),
```

```
      eta = qlogis(p_mean) +
        beta_text_jaccard * text_length +
        u_record,
      mu = plogis(eta),
      shape1 = mu * phi,
      shape2 = (1 - mu) * phi,
      jaccard_avg = rbeta(n(), shape1 = shape1, shape2 = shape2)
    )

  # Match the main analysis boundary adjustment exactly.
  n_jaccard <- nrow(out)

  out |>
    mutate(
      jaccard_beta =
        (jaccard_avg * (n_jaccard - 1) + 0.5) / n_jaccard
    ) |>
    select(record, condition, group, text_length, jaccard_avg, jaccard_beta)
}

fit_jaccard_model <- function(data) {
  glmmTMB::glmmTMB(
    jaccard_beta ~ condition * group + text_length + (1 | record),
    family = glmmTMB::beta_family(link = "logit"),
    data = data
  )
}

# Q2 target: three coder-group comparisons averaged over condition.
# Tukey adjustment is applied to the family of three comparisons.
extract_jaccard_group_pvalues <- function(fit) {
  emm_group <- emmeans::emmeans(fit, ~group, type = "response")

  emmeans::contrast(
    emm_group,
    method = "pairwise",
    adjust = "tukey"
  ) |>
    summary() |>
    as.data.frame() |>
    transmute(
      contrast = as.character(contrast),
      p_value = p.value
    )
}

# Q1 target: one AI-versus-manual comparison averaged over coder groups.
extract_jaccard_ai_pvalue <- function(fit) {
  emm_condition <- emmeans::emmeans(
    fit,
    ~condition,
    type = "response"
  )
```

```
  emmeans::contrast(
    emm_condition,
    method = "pairwise",
    adjust = "none"
  ) |>
    summary() |>
    as.data.frame() |>
    pull(p.value) |>
    first()
}

# One simulation wrapper is used by both Jaccard questions.
run_jaccard_once <- function(N, delta_AI, target = c("group", "AI")) {
  target <- match.arg(target)

  tryCatch(
    {
      data <- simulate_jaccard_data(N = N, delta_AI = delta_AI)
      fit <- suppressMessages(suppressWarnings(fit_jaccard_model(data)))

      if (target == "group") {
        extract_jaccard_group_pvalues(fit)
      } else {
        extract_jaccard_ai_pvalue(fit)
      }
    },
    error = function(e) {
      if (target == "group") {
        tibble(
          contrast = c(
            "Clinician - Coder",
            "Clinician - Student",
            "Coder - Student"
          ),
          p_value = NA_real_
        )
      } else {
        NA_real_
      }
    }
  )
}
```

4 Question 1: AI effect on Jaccard agreement

Does marginal Jaccard agreement differ between AI-assisted and manual coding? The target is the AI-versus-manual contrast averaged equally over coder groups, matching `emmeans(fit, ~ condition, type = "response")` followed by `pairs()`.

```
q1_results <- progressr::with_progress({
  progress <- progressr::progressor(along = N_grid)

  purrr::map_dfr(N_grid, function(N) {
```

```
    message("Q1: starting N = ", N)

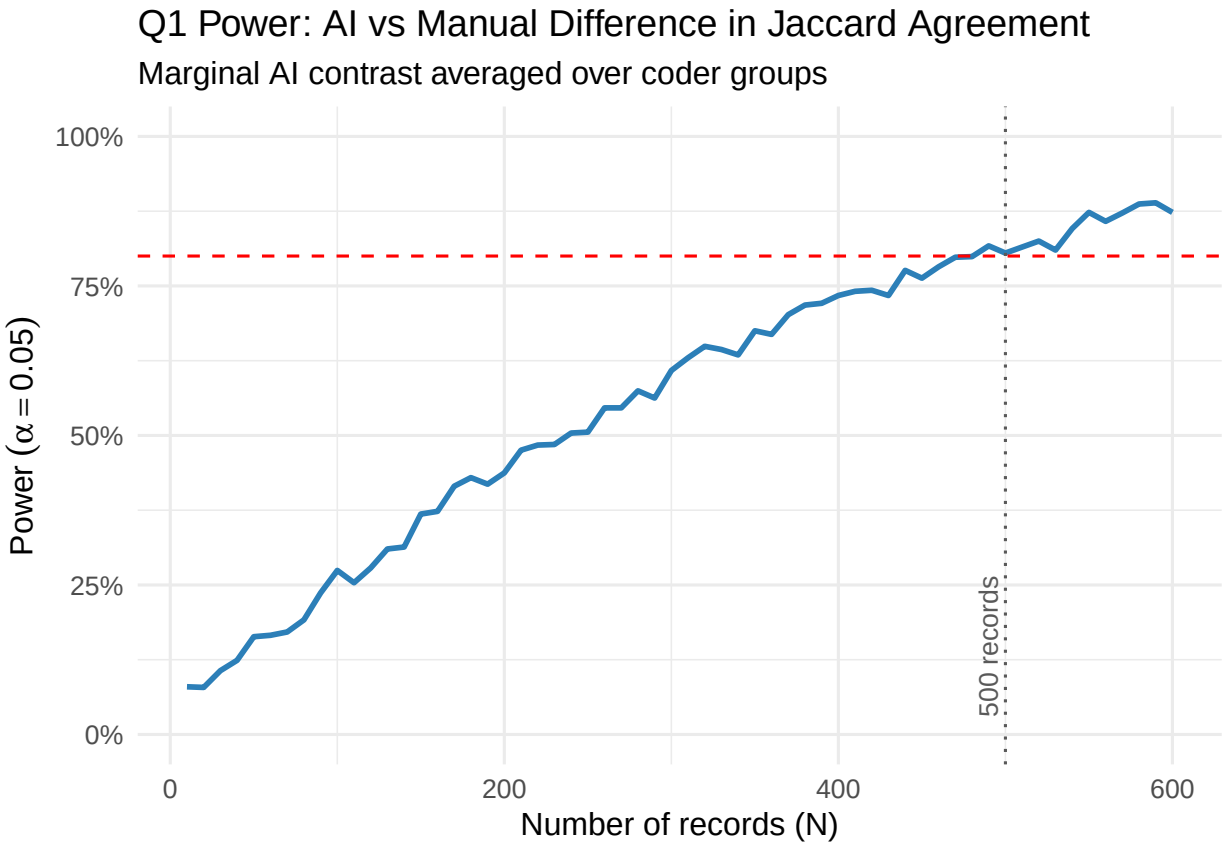
    p_values <- furrr::future_map_dbl(
      seq_len(reps),
      function(i) {
        run_jaccard_once(
          N = N,
          delta_AI = 0.05,
          target = "AI"
        )
      },
      .options = parallel_options
    )

    result <- tibble(
      N = N,
      successful_fits = sum(!is.na(p_values)),
      power = mean(p_values < alpha, na.rm = TRUE)
    )

    progress(message = paste0("Q1 completed N = ", N))
    result
  })
})

q1_plot <- ggplot(q1_results, aes(x = N, y = power)) +
  geom_line(color = "#2c7fb8", linewidth = 1) +
  labs(
    title = "Q1 Power: AI vs Manual Difference in Jaccard Agreement",
    subtitle = "Marginal AI contrast averaged over coder groups",
    x = "Number of records (N)",
    y = expression(Power ~ (alpha == 0.05))
  )

print(add_power_references(q1_plot))
```




```
report_minimum_N(q1_results, "Q1")
```

```
## # A tibble: 1 x 2
##   question minimum_N
##   <chr>        <dbl>
## 1 Q1             490
```

5 Question 2: Between-group Jaccard agreement

Does marginal Jaccard agreement vary between coder groups? This analysis estimates power separately for the three pairwise group contrasts averaged over AI condition. The p-values are Tukey-adjusted within each simulated dataset.

For this question, the AI lift remains zero so that the generating signal is the prespecified difference among manual-condition group means.

```
q2_results <- progressr::with_progress({
  progress <- progressr::progressor(along = N_grid)

  purrr::map_dfr(N_grid, function(N) {
    message("Q2: starting N = ", N)

    sim_results <- furrr::future_map_dfr(
      seq_len(reps),
      function(i) {
        run_jaccard_once(
          N = N,
          delta_AI = 0.00,
```

```
          target = "group"
        )
      },
      .options = parallel_options
    )

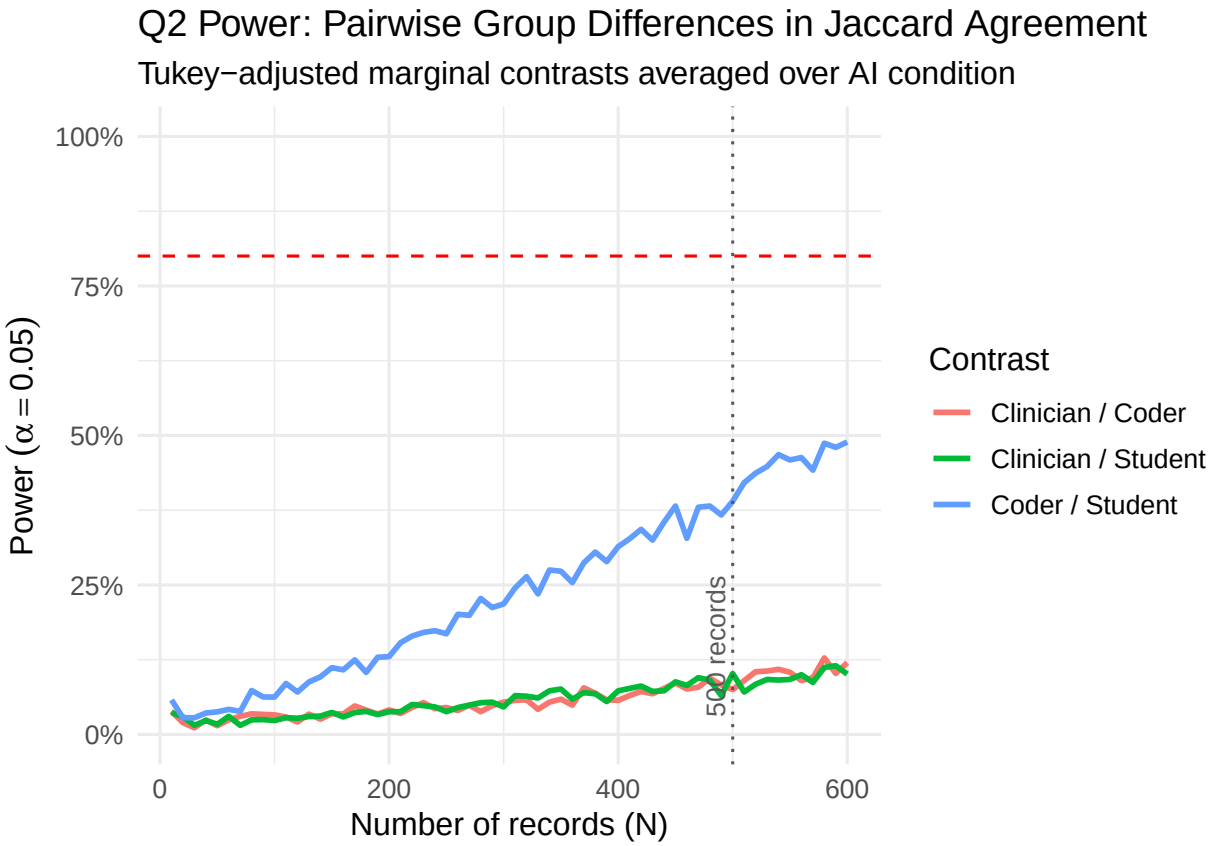
    result <- sim_results |>
      group_by(contrast) |>
      summarise(
        N = N,
        successful_fits = sum(!is.na(p_value)),
        power = mean(p_value < alpha, na.rm = TRUE),
        .groups = "drop"
      )

    progress(message = paste0("Q2 completed N = ", N))
    result
  })
})

if (all(q2_results$successful_fits == 0)) {
  stop(
    "All Q2 model fits or contrast extractions failed. ",
    "Inspect run_jaccard_once() without tryCatch for one replicate."
  )
}

q2_plot <- ggplot(
  q2_results,
  aes(x = N, y = power, color = contrast)
) +
  geom_line(linewidth = 1) +
  labs(
    title = "Q2 Power: Pairwise Group Differences in Jaccard Agreement",
    subtitle = "Tukey-adjusted marginal contrasts averaged over AI condition",
    x = "Number of records (N)",
    y = expression(Power ~ (alpha == 0.05)),
    color = "Contrast"
  )

print(add_power_references(q2_plot))
```




```
report_minimum_N(q2_results, "Q2")
```

```
## # A tibble: 3 x 3
##   question contrast            minimum_N
##   <chr>    <chr>                   <int>
## 1 Q2       Clinician / Coder          NA
## 2 Q2       Clinician / Student        NA
## 3 Q2       Coder / Student            NA
```

6 Coding-time simulation

6.1 Model and assumptions

For record r and coder i, coding time is generated on the log scale:

$$\log(T_{ri}) = \alpha_0 + \alpha_{AI} AI_r + \alpha_{Group} Group_i + \gamma Order_{ri} + \beta_L L_r + b_i + u_r + \epsilon_{ri}.$$

The coder roster contains eight coders with the study allocation of three clinicians, two coders, and three students. Each coder receives an independent random ordering of the N records. `order_global` is therefore the uncentered integer sequence from 1 through N within each coder, matching the construction in the study analysis.

The fitted model is:

```
log(time_sec) ~ condition * group + order_global + text_length +
  (1 | coder_id) + (1 | record)
```

The pre-hoc assumptions are retained. Because the original power script did not specify a text-length effect, `beta_text_time` is set to zero during generation while text length remains in the fitted model. No 600-second exclusion is applied.

```
# Pre-hoc fixed-effect assumptions.
gm_manual_coder <- 120
ratio_student <- 1.25
ratio_clinician <- 1.10
learn_slope <- -0.002
beta_text_time <- 0

# Pre-hoc random-effect assumptions.
sd_coder_time <- 0.20
sd_record_time <- 0.50

# The original script used different residual assumptions for the group and AI questions.
sd_resid_time_group <- 0.20
sd_resid_time_ai <- 0.35
```

6.2 One coding-time data-generating function

```
simulate_time_data <- function(N, ai_ratio, sd_resid) {
  record_data <- tibble(
    record = factor(seq_len(N)),
    condition = assign_condition(N),
    text_length = simulate_text_length(N),
    b_record = rnorm(N, mean = 0, sd = sd_record_time)
  )

  coder_data <- coder_roster |>
    mutate(
      b_coder = rnorm(n(), mean = 0, sd = sd_coder_time)
    )

  # Create an independent randomized task order for each coder.
  order_data <- tidyr::crossing(
    coder_id = coder_data$coder_id,
    record = factor(seq_len(N), levels = levels(record_data$record))
  ) |>
    group_by(coder_id) |>
    mutate(
      simulated_order = sample.int(N, size = N, replace = FALSE)
    ) |>
    arrange(simulated_order, .by_group = TRUE) |>
    mutate(order_global = row_number()) |>
    ungroup() |>
    select(coder_id, record, order_global)

  group_ratio <- c(
    Student = ratio_student,
    Clinician = ratio_clinician,
    Coder = 1.00
  )
```

```r
  condition_ratio <- c(
    Manual = 1.00,
    AI = ai_ratio
  )

  tidyr::crossing(
    coder_id = coder_data$coder_id,
    record = factor(seq_len(N), levels = levels(record_data$record))
  ) |>
    left_join(coder_data, by = "coder_id") |>
    left_join(record_data, by = "record") |>
    left_join(order_data, by = c("coder_id", "record")) |>
    mutate(
      expected_log_time =
        log(gm_manual_coder) +
          log(unname(group_ratio[as.character(group)])) +
          log(unname(condition_ratio[as.character(condition)])) +
          learn_slope * order_global +
          beta_text_time * text_length +
          b_coder +
          b_record,
      log_time = expected_log_time +
        rnorm(n(), mean = 0, sd = sd_resid),
      time_sec = exp(log_time)
    ) |>
    select(
      record,
      coder_id,
      group,
      condition,
      order_global,
      text_length,
      time_sec,
      log_time
    )
}

fit_time_model <- function(data) {
  lmerTest::lmer(
    log(time_sec) ~ condition * group + order_global + text_length +
      (1 | coder_id) + (1 | record),
    data = data,
    REML = TRUE
  )
}

# Q3 target: one AI-versus-manual comparison averaged over coder groups.
extract_time_ai_pvalue <- function(fit) {
  emm_condition <- emmeans::emmeans(
    fit,
    ~condition,
    type = "response",
    lmer.df = "satterthwaite"
```

```
  )

  emmeans::contrast(
    emm_condition,
    method = "pairwise",
    adjust = "none"
  ) |>
    summary() |>
    as.data.frame() |>
    pull(p.value) |>
    first()
}

# Q4 target: three coder-group comparisons averaged over condition.
# Tukey adjustment is applied to the family of three comparisons.
extract_time_group_pvalues <- function(fit) {
  emm_group <- emmeans::emmeans(
    fit,
    ~group,
    type = "response",
    lmer.df = "satterthwaite"
  )

  emmeans::contrast(
    emm_group,
    method = "pairwise",
    adjust = "tukey"
  ) |>
    summary() |>
    as.data.frame() |>
    transmute(
      contrast = as.character(contrast),
      p_value = p.value
    )
}

# One simulation wrapper is used by both coding-time questions.
run_time_once <- function(
  N,
  ai_ratio,
  sd_resid,
  target = c("group", "AI")
) {
  target <- match.arg(target)

  tryCatch(
    {
      data <- simulate_time_data(
        N = N,
        ai_ratio = ai_ratio,
        sd_resid = sd_resid
      )
```

```
      fit <- suppressMessages(suppressWarnings(fit_time_model(data)))

      if (target == "group") {
        extract_time_group_pvalues(fit)
      } else {
        extract_time_ai_pvalue(fit)
      }
    },
    error = function(e) {
      if (target == "group") {
        tibble(
          contrast = c(
            "Clinician - Coder",
            "Clinician - Student",
            "Coder - Student"
          ),
          p_value = NA_real_
        )
      } else {
        NA_real_
      }
    }
  )
}
```

7 Question 3: AI effect on coding time

Does marginal coding time differ between AI-assisted and manual coding? The target is the AI-versus-manual contrast averaged equally over coder groups, matching `emmeans(fit, ~ condition, type = "response")` followed by a single unadjusted pairwise contrast.

```
q3_results <- progressr::with_progress({
  progress <- progressr::progressor(along = N_grid)

  purrr::map_dfr(N_grid, function(N) {
    message("Q3: starting N = ", N)

    p_values <- furrr::future_map_dbl(
      seq_len(reps),
      function(i) {
        run_time_once(
          N = N,
          ai_ratio = 0.80,
          sd_resid = sd_resid_time_ai,
          target = "AI"
        )
      },
      .options = parallel_options
    )

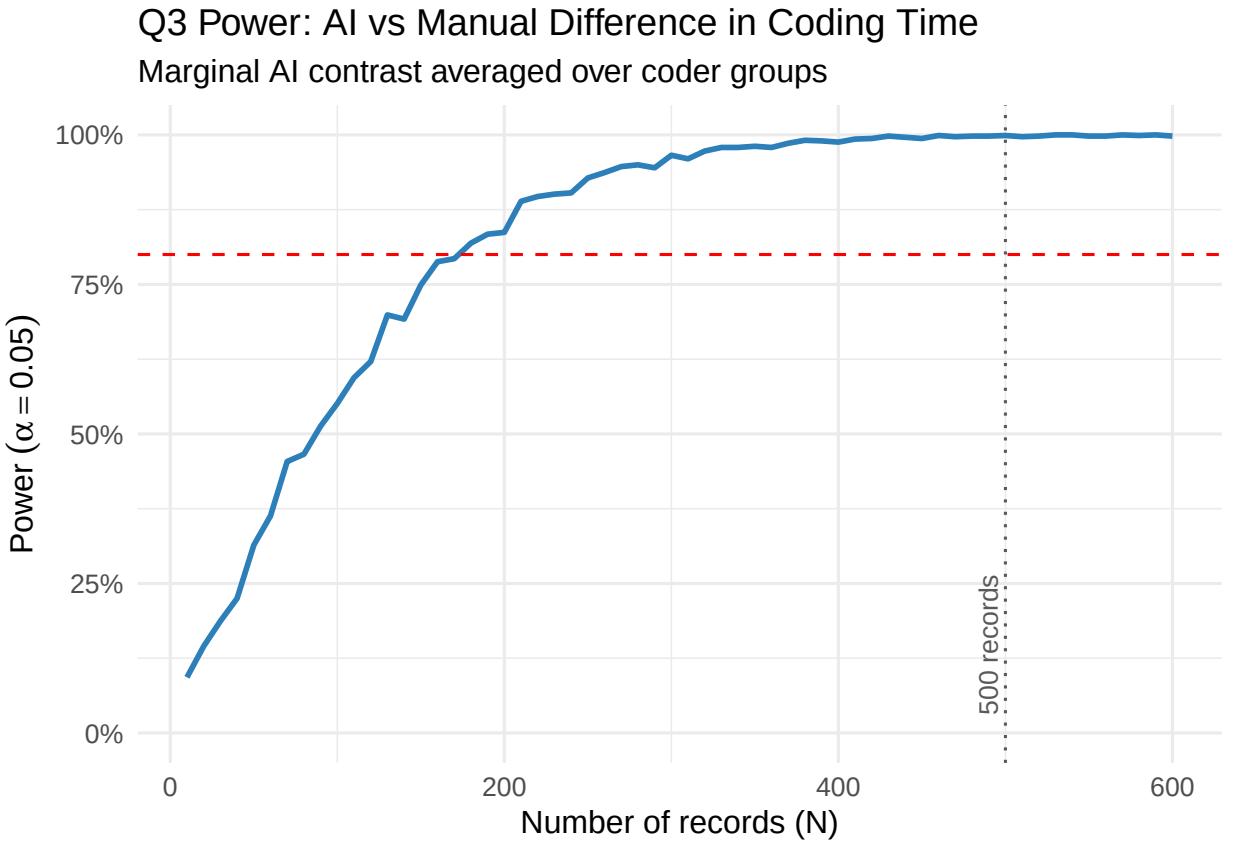
    result <- tibble(
      N = N,
      successful_fits = sum(!is.na(p_values)),
```

```
      power = mean(p_values < alpha, na.rm = TRUE)
    )

    progress(message = paste0("Q3 completed N = ", N))
    result
  })
})

q3_plot <- ggplot(q3_results, aes(x = N, y = power)) +
  geom_line(color = "#2c7fb8", linewidth = 1) +
  labs(
    title = "Q3 Power: AI vs Manual Difference in Coding Time",
    subtitle = "Marginal AI contrast averaged over coder groups",
    x = "Number of records (N)",
    y = expression(Power ~ (alpha == 0.05))
  )

print(add_power_references(q3_plot))
```




```
report_minimum_N(q3_results, "Q3")
```

```
## # A tibble: 1 x 2
##   question minimum_N
##   <chr>        <dbl>
## 1 Q3             180
```

8 Question 4: Between-group coding time

Does marginal coding time vary among coder groups? This analysis estimates power separately for the three pairwise group contrasts averaged over AI condition. The p-values are Tukey-adjusted within each simulated dataset. For this question, `ai_ratio = 1.00`, so no AI effect is generated.

```
q4_results <- progressr::with_progress({
  progress <- progressr::progressor(along = N_grid)

  purrr::map_dfr(N_grid, function(N) {
    message("Q4: starting N = ", N)

    sim_results <- furrr::future_map_dfr(
      seq_len(reps),
      function(i) {
        run_time_once(
          N = N,
          ai_ratio = 1.00,
          sd_resid = sd_resid_time_group,
          target = "group"
        )
      },
      .options = parallel_options
    )

    result <- sim_results |>
      group_by(contrast) |>
      summarise(
        N = N,
        successful_fits = sum(!is.na(p_value)),
        power = mean(p_value < alpha, na.rm = TRUE),
        .groups = "drop"
      )

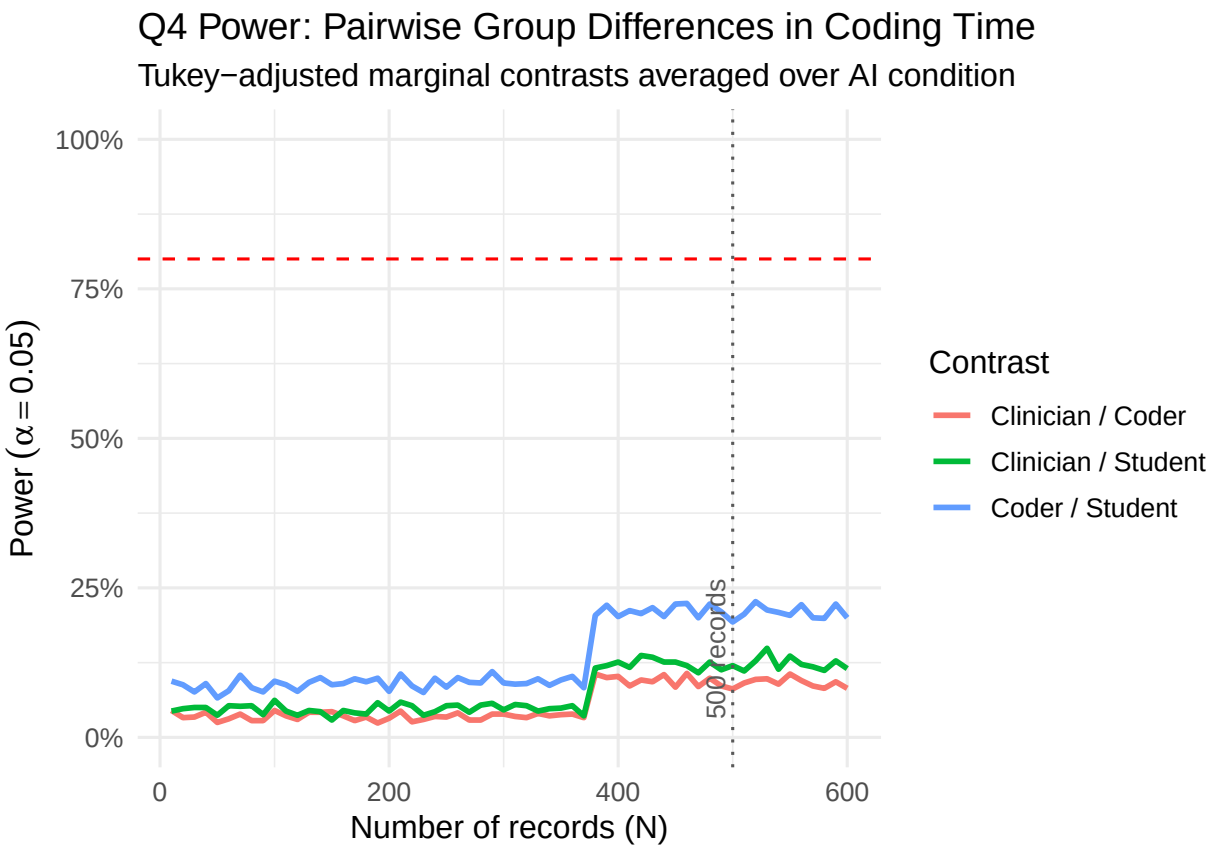
    progress(message = paste0("Q4 completed N = ", N))
    result
  })
})

if (all(q4_results$successful_fits == 0)) {
  stop(
    "All Q4 model fits or contrast extractions failed. ",
    "Inspect run_time_once() without tryCatch for one replicate."
  )
}

q4_plot <- ggplot(
  q4_results,
  aes(x = N, y = power, color = contrast)
) +
  geom_line(linewidth = 1) +
  labs(
    title = "Q4 Power: Pairwise Group Differences in Coding Time",
    subtitle = "Tukey-adjusted marginal contrasts averaged over AI condition",
```

```
    x = "Number of records (N)",
    y = expression(Power ~ (alpha == 0.05)),
    color = "Contrast"
  )

print(add_power_references(q4_plot))
```




```
report_minimum_N(q4_results, "Q4")
```

```
## # A tibble: 3 x 3
##   question contrast            minimum_N
##   <chr>    <chr>                   <int>
## 1 Q4       Clinician / Coder          NA
## 2 Q4       Clinician / Student        NA
## 3 Q4       Coder / Student            NA
```

```
future::plan(future::sequential)
```